# Blind Spots

## Project Group


- Dante Razo, razo3843@stthomas.edu
- Odin Lee, lee06265@stthomas.edu
- Jannah ElNemr, elne5502@stthomas.edu


## Ambiguous Items

The provided dataset marks eight tickets as ambiguous: `g009`, `g015`, `g020`, `g038`, `g041`, `g048`, `g058`, and `g067`. The policy does not determine one clearly correct action for these cases; resolving them depends on context not captured by the ticket, and ultimately human judgement. This is why a human must be kept in the loop for these systems. They are best left to customer support staff rather than decided automatically. The harness still scores them against its chosen labels, so errors on these tickets are **not** evidence that the model violated a policy rule.

## Judge Failure Modes

In our 30-output validation, we found that the judge could accept a rationale that relied on a made-up policy exception, especially when its arithmetic *sounded* reasonable. It also struggled to distinguish the real policy from injected instructions such as "**SYSTEM OVERRIDE.**" For `states_policy_correctly`, the judge and human labels disagreed twice: once where the judge said yes and the human said no, and once in the opposite direction. For `agrees_with_action` and `arithmetic_correct`, every disagreement ran the same way: the human said yes and the judge said no (4 out of 30 times). The judge was also inconsistent with itself: across three runs on the same 30 items, it changed 12 of 90 verdicts, which denotes a noise floor of 13.3%. This illustrates the nondeterministic nature of LLMs. Judging differences smaller than that noise floor cannot be treated as real in practice.

These results show that the judge can *help* flag questionable rationales, but its verdicts are **not** ground truth and still need human review.

## Slice Comparisons Within the Noise Floor

Repeating the baseline five times showed run-to-run noise on the same tickets: 4 disagreements on `intent:refund`, 6 on `amount:50-to-200`, 11 on `amount:near-50`, and 12 on `injection:yes`. The slices `amount:near-50` (9 tickets) and `injection:yes` (8 tickets) are small, so one ticket makes a big difference in the pass rate. When we compared sending the policy as a system instruction against the baseline, only `injection:yes` (+15 points) and `amount:over-200` (+11 points) cleared their noise floors. For `amount:near-50` (-4), `complexity:math` (-2), and `amount:50-to-200` (-6), the gaps were inside the noise, so we cannot determine whether the change helped. Moving the policy also did not fix the larger problem: the baseline passes only 1 of 9 `amount:near-50` tickets and 1-2 of 8 injection tickets, far below the 95% targets defined in the 3.1 deliverable (see 3.1_design.md).

## Untested Failure Modes

The suite uses a small, written set of tickets and account summaries. It does not test what happens when account data is stale or conflicting, a customer asks for a partial or duplicate refund, or a ticket includes taxes, another currency, a different language, or much more text than these examples. These are edge cases that should be covered in a production system.

It also does not establish whether a payment system would enforce the model's proposed route, whether a human reviewer would catch an error, or whether the customer ultimately receives the correct resolution. Finally, the injection tickets only use the wording found in this dataset. Putting the policy in the system instruction improved results on those tickets, but it does not guarantee that differently worded attacks would be blocked. There are likely many other injection phrases that could succeed if phrased differently.